Air University

Operating Systems (Lab)

Year 2026

Ms. Qurrat Ul Ain

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: June 05, 2026

Lab # 13

Environment Setup:

I created a new directory named "Lab13" by executing the **mkdir Lab13** command, and then I successfully navigated into that directory using the **cd Lab13** command.

```
usama@ubuntu:~$ mkdir Lab13
usama@ubuntu:~$ cd Lab13
```

TASK 1 — Implement the Provided C Code

File creation & Compilation: I created and opened a new C source file named "task1.c" using the **nano** text editor, then compiled it into an executable named "task1" using the **gcc** compiler.

```
usama@ubuntu:~/Lab13$ nano task1.c
usama@ubuntu:~/Lab13$ gcc task1.c -o task1
```

Code:

```c
#include <stdio.h>

#define maxProcesses 20
#define maxResources 10

int main()
{
    int mark[maxProcesses];
    int r[maxResources];
    int alloc[maxProcesses][maxResources];
    int request[maxProcesses][maxResources];
    int avail[maxResources];
    int w[maxResources];
    int np, nr, i, j;

    // Initialize mark array to 0
    for (i = 0; i < maxProcesses; i++)
        mark[i] = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &np);

    printf("Enter the number of resources: ");
    scanf("%d", &nr);

    printf("Enter the total amount of each resource:\n");
    for (i = 0; i < nr; i++)
    {
        printf("Total Amount of Resource R %d: ", i + 1);
        scanf("%d", &r[i]);
    }
```

```
    printf("Enter the request matrix:\n");
    for (i = 0; i < np; i++)
        for (j = 0; j < nr; j++)
            scanf("%d", &request[i][j]);

    printf("Enter the allocation matrix:\n");
    for (i = 0; i < np; i++)
        for (j = 0; j < nr; j++)
            scanf("%d", &alloc[i][j]);

    // Available Resource calculation
    for (j = 0; j < nr; j++)
    {
        avail[j] = r[j];
        for (i = 0; i < np; i++)
            avail[j] -= alloc[i][j];
    }

    // Mark processes with zero allocation
    for (i = 0; i < np; i++)
    {
        int count = 0;
        for (j = 0; j < nr; j++)
        {
            if (alloc[i][j] == 0)
                count++;
            else
                break;
        }
        if (count == nr)
            mark[i] = 1;
    }

    // Initialize W with avail
    for (j = 0; j < nr; j++)
        w[j] = avail[j];

    // Mark processes with request less than or equal to W
    for (i = 0; i < np; i++)
    {
        int canBeProcessed = 0;
        if (mark[i] != 1)
        {
            for (j = 0; j < nr; j++)
            {
                if (request[i][j] <= w[j])
                    canBeProcessed = 1;
                else
                {
                    canBeProcessed = 0;
                    break;
                }
```

```
            }
            if (canBeProcessed)
            {
                mark[i] = 1;
                for (j = 0; j < nr; j++)
                    w[j] += alloc[i][j];
            }
        }
    }

    // Checking for unmarked processes
    int deadlock = 0;
    for (i = 0; i < np; i++)
    {
        if (mark[i] != 1)
        {
            deadlock = 1;
            break;
        }
    }

    if (deadlock)
        printf("\nDeadlock detected\n");
    else
        printf("\nNo Deadlock possible\n");

    return 0;
}
```

Output: I executed the compiled program using **./task1** and provided the required inputs—the number of processes and resources, the total resource amounts, and the request and allocation matrices—to perform a deadlock detection analysis, which concluded that no deadlock is possible in this scenario.

```
usama@ubuntu:~/Lab13$ ./task1
Enter the number of processes: 3
Enter the number of resources: 4
Enter the total amount of each resource:
Total Amount of Resource R 1: 10
Total Amount of Resource R 2: 5
Total Amount of Resource R 3: 7
Total Amount of Resource R 4: 8
Enter the request matrix:
1 2 2 2
2 0 1 1
0 1 0 2
Enter the allocation matrix:
0 1 0 1
1 0 1 2
1 2 1 0

No Deadlock possible
```

TASK 2 — Safe State Order (C++ Program)

File creation & Compilation: I created and opened a new C++ source file named "task2.cpp" using the **nano** text editor, then compiled it into an executable named "task2" using the **g++** compiler.

```
usama@ubuntu:~/Lab13$ nano task2.cpp
usama@ubuntu:~/Lab13$ g++ task2.cpp -o task2
```

Code:

```cpp
#include <iostream>
#include <vector>
using namespace std;

#define maxProcesses 20
#define maxResources 10

int main()
{
    int mark[maxProcesses] = {0};
    int r[maxResources];
    int alloc[maxProcesses][maxResources];
    int request[maxProcesses][maxResources];
    int avail[maxResources];
    int w[maxResources];
    int np, nr, i, j;

    cout << "Enter the number of processes: ";
    cin >> np;

    cout << "Enter the number of resources: ";
    cin >> nr;

    cout << "Enter the total amount of each resource:\n";
    for (i = 0; i < nr; i++)
    {
        cout << "Total Amount of Resource R " << i + 1 << ": ";
        cin >> r[i];
    }

    cout << "Enter the request matrix:\n";
    for (i = 0; i < np; i++)
        for (j = 0; j < nr; j++)
            cin >> request[i][j];

    cout << "Enter the allocation matrix:\n";
    for (i = 0; i < np; i++)
        for (j = 0; j < nr; j++)
            cin >> alloc[i][j];

    // Calculate available resources
    for (j = 0; j < nr; j++)
    {
```

```
        avail[j] = r[j];
        for (i = 0; i < np; i++)
            avail[j] -= alloc[i][j];
    }

    // Mark processes with zero allocation
    for (i = 0; i < np; i++)
    {
        int count = 0;
        for (j = 0; j < nr; j++)
        {
            if (alloc[i][j] == 0)
                count++;
            else
                break;
        }
        if (count == nr)
            mark[i] = 1;
    }

    // Initialize work vector
    for (j = 0; j < nr; j++)
        w[j] = avail[j];

    // Find safe sequence using repeated passes
    vector<int> safeOrder;
    int found = 1;

    while (found)
    {
        found = 0;
        for (i = 0; i < np; i++)
        {
            if (mark[i] != 1)
            {
                int canBeProcessed = 1;
                for (j = 0; j < nr; j++)
                {
                    if (request[i][j] > w[j])
                    {
                        canBeProcessed = 0;
                        break;
                    }
                }
                if (canBeProcessed)
                {
                    mark[i] = 1;
                    safeOrder.push_back(i);
                    for (j = 0; j < nr; j++)
                        w[j] += alloc[i][j];
                    found = 1;
                }
```

```
            }
        }
    }

    // Check for deadlock
    int deadlock = 0;
    for (i = 0; i < np; i++)
    {
        if (mark[i] != 1)
        {
            deadlock = 1;
            break;
        }
    }

    if (deadlock)
    {
        cout << "\nDeadlock detected" << endl;
        cout << "Deadlocked processes: ";
        for (i = 0; i < np; i++)
            if (mark[i] != 1)
                cout << "P" << i + 1 << " ";
        cout << endl;
    }
    else
    {
        cout << "\nNo Deadlock possible" << endl;
        cout << "Safe execution order: ";
        for (int k = 0; k < (int)safeOrder.size(); k++)
        {
            cout << "P" << safeOrder[k] + 1;
            if (k < (int)safeOrder.size() - 1)
                cout << " -> ";
        }
        cout << endl;
    }

    return 0;
}
```

Output: I executed the task2 program twice with different inputs to test its functionality. In the first instance, the system was found to be in a safe state with a safe execution order of $P1 \rightarrow P2 \rightarrow P3$. In the second instance, I provided inputs that resulted in the program successfully detecting a deadlock involving processes *P1* and *P2*.

```
usama@ubuntu:~/Lab13$ ./task2
Enter the number of processes: 3
Enter the number of resources: 4
Enter the total amount of each resource:
Total Amount of Resource R 1: 10
Total Amount of Resource R 2: 5
Total Amount of Resource R 3: 7
Total Amount of Resource R 4: 8
Enter the request matrix:
1 2 2 2
2 0 1 1
0 1 0 2
Enter the allocation matrix:
0 1 0 1
1 0 1 2
1 2 1 0

No Deadlock possible
Safe execution order: P1 -> P2 -> P3
```

```
usama@ubuntu:~/Lab13$ ./task2
Enter the number of processes: 2
Enter the number of resources: 2
Enter the total amount of each resource:
Total Amount of Resource R 1: 1
Total Amount of Resource R 2: 1
Enter the request matrix:
0 1
1 0
Enter the allocation matrix:
1 0
0 1

Deadlock detected
Deadlocked processes: P1 P2
```

TASK 3 — Modified Code with Row-by-Row Matrix Input

File Creation & Compilation: I created and opened a new C++ source file named "task3.cpp" using the **nano** text editor, and then I compiled this source file into an executable named "task3" using the **g++** compiler.

```
usama@ubuntu:~/Lab13$ nano task3.cpp
usama@ubuntu:~/Lab13$ g++ task3.cpp -o task3
```

Code:

```cpp
#include <iostream>
using namespace std;

#define maxProcesses 20
#define maxResources 10

int main()
{
    int mark[maxProcesses] = {0};
    int r[maxResources];
    int alloc[maxProcesses][maxResources];
    int request[maxProcesses][maxResources];
    int avail[maxResources];
    int w[maxResources];
    int np, nr, i, j;

    cout << "Enter the number of processes: ";
    cin >> np;

    cout << "Enter the number of resources: ";
    cin >> nr;

    cout << "\nEnter the total amount of each resource:\n";
    for (i = 0; i < nr; i++)
    {
        cout << "Total Amount of Resource R " << i + 1 << ": ";
        cin >> r[i];
    }

    cout << "\nEnter the request matrix:\n";
    for (i = 0; i < np; i++)
        for (j = 0; j < nr; j++)
            cin >> request[i][j];

    cout << "Enter the allocation matrix:\n";
    for (i = 0; i < np; i++)
        for (j = 0; j < nr; j++)
            cin >> alloc[i][j];

    // Available Resource calculation
    for (j = 0; j < nr; j++)
    {
        avail[j] = r[j];
```

```
        for (i = 0; i < np; i++)
            avail[j] -= alloc[i][j];
    }

    // Mark processes with zero allocation
    for (i = 0; i < np; i++)
    {
        int count = 0;
        for (j = 0; j < nr; j++)
        {
            if (alloc[i][j] == 0)
                count++;
            else
                break;
        }
        if (count == nr)
            mark[i] = 1;
    }

    // Initialize W with avail
    for (j = 0; j < nr; j++)
        w[j] = avail[j];

    // Mark processes with request less than or equal to W
    for (i = 0; i < np; i++)
    {
        int canBeProcessed = 0;
        if (mark[i] != 1)
        {
            for (j = 0; j < nr; j++)
            {
                if (request[i][j] <= w[j])
                    canBeProcessed = 1;
                else
                {
                    canBeProcessed = 0;
                    break;
                }
            }
            if (canBeProcessed)
            {
                mark[i] = 1;
                for (j = 0; j < nr; j++)
                    w[j] += alloc[i][j];
            }
        }
    }

    // Checking for unmarked processes
    int deadlock = 0;
    for (i = 0; i < np; i++)
    {
```

```
        if (mark[i] != 1)
        {
            deadlock = 1;
            break;
        }
    }

    if (deadlock)
        cout << "\nDeadlock detected" << endl;
    else
        cout << "\nNo Deadlock possible" << endl;

    return 0;
}
```

Output: I executed the **task3** program twice to test its deadlock detection capabilities. In the first execution, I provided inputs that resulted in the program determining that no deadlock is possible in the system. In the second execution, I provided a different set of inputs that led the program to successfully identify and report that a deadlock is present.

```
usama@ubuntu:~/Lab13$ ./task3
Enter the number of processes: 3
Enter the number of resources: 4

Enter the total amount of each resource:
Total Amount of Resource R 1: 10
Total Amount of Resource R 2: 5
Total Amount of Resource R 3: 7
Total Amount of Resource R 4: 8

Enter the request matrix:
1 2 2 1
2 0 1 1
0 1 0 2
Enter the allocation matrix:
0 1 0 1
1 0 1 2
1 2 1 0

No Deadlock possible
```

```
usama@ubuntu:~/Lab13$ ./task3
Enter the number of processes: 3
Enter the number of resources: 2

Enter the total amount of each resource:
Total Amount of Resource R 1: 2
Total Amount of Resource R 2: 2

Enter the request matrix:
1 1
1 0
0 1
Enter the allocation matrix:
1 0
1 0
0 1

Deadlock detected
```

- End of Lab Assignment -